IERG4320/IEMS5726: Data Science in Practice

Lecture 7: Data Modeling II

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

Outline

1. Clustering Algorithm and its Evaluation
2. Reinforcement Learning Algorithm and its Evaluation

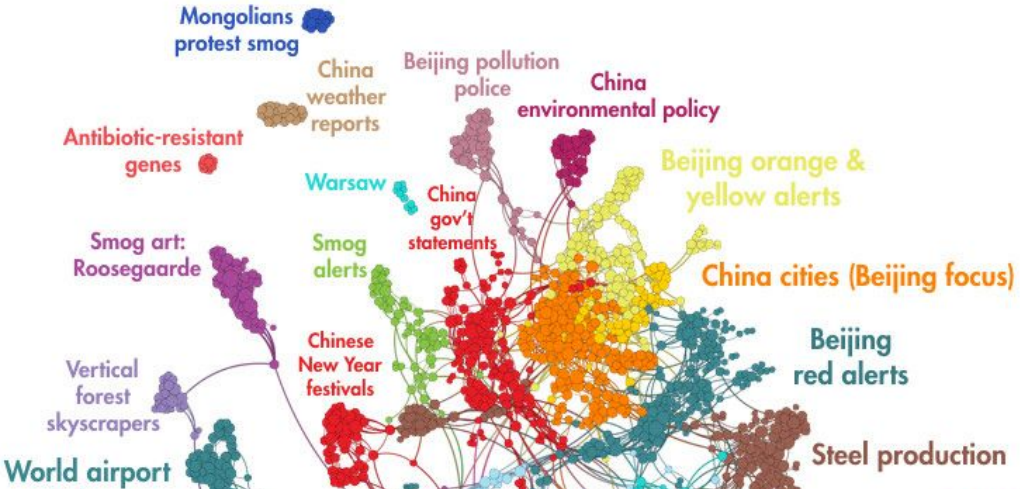
All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

Clustering

- Clustering is often used to find appropriate **groupings** of elements for a set of data.
- The data in each group should be **similar** to one another.
- Automatic cluster detection is rarely used in isolation.
- Once clusters have been detected, other methods must be applied in order to figure out what the clusters mean.
- It is a kind of **undirected knowledge discovery or unsupervised learning**.

Unsupervised Learning / Clustering

- To identify hidden patterns in a dataset without labels
- Examples:
- Understand what products are usually bought together (e.g. Basket Analysis)
- Understand major topics of news articles (e.g. News Clustering)
- Identifying communities in social networks
- Clustering can be used to perform dimensionality reduction



Types of Clusters

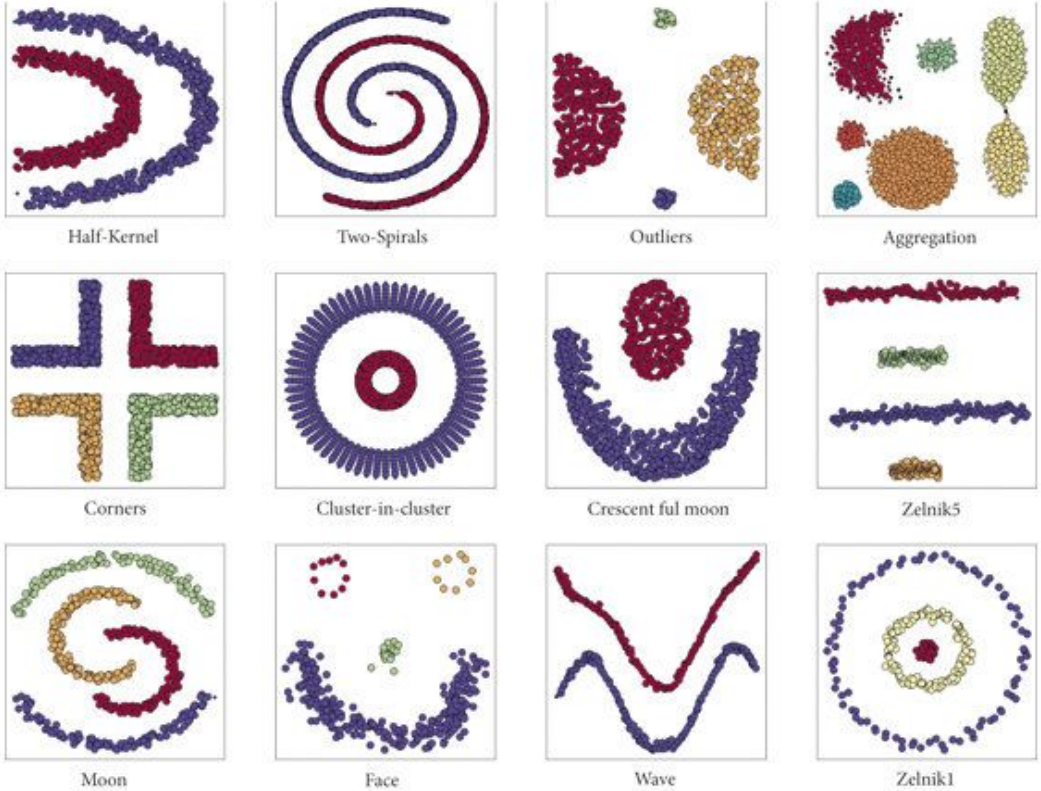


Image credit: ResearchGate

Some Top Clustering Algorithms

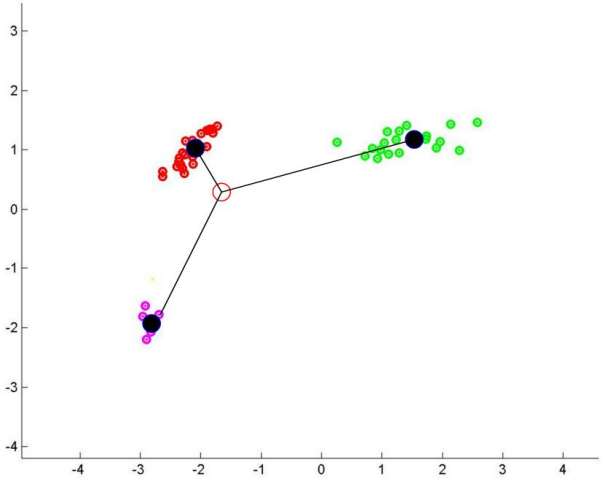
1. K-Means
2. Agglomerative Clustering
3. Spectral Clustering

Reference: https://machinelearningmastery.com/clustering-algorithms-with-python/

k-Means Clustering

- The basic algorithm is very simple
- Suppose that we have n example feature vectors $x_1, x_2, ..., x_n$
- Number of clusters, k, must be specified (k < n) .
- Each cluster is associated with a centroid (center point or the mean of the vectors), m_i , in cluster i.
- Each point is assigned to the cluster with the closest centroid
- Parameters include:

1. k: number of clusters
2. n_init: number of initializations to perform
    - Some random points, farthest points, centroids of some points, … etc



k-Means Algorithm

1. Make initial guesses for the means m1, m2, ..., mk
2. Until there is no change in any mean
   - Assign each data point to the cluster whose mean is the nearest.
   - Calculate the mean of each cluster.
   - For i from 1 to k
     - Replace mi with the mean of all examples for Cluster i.
   - End for
3. End until

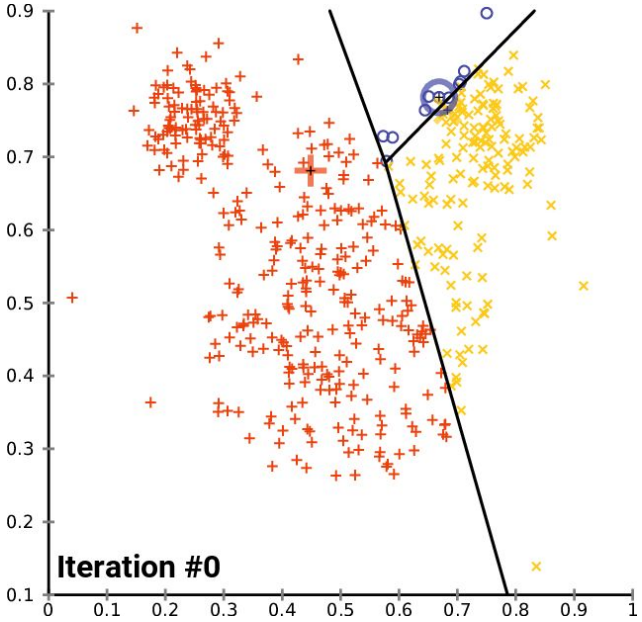


Image credit: Wiki

More Illustrations on k-Means Clustering

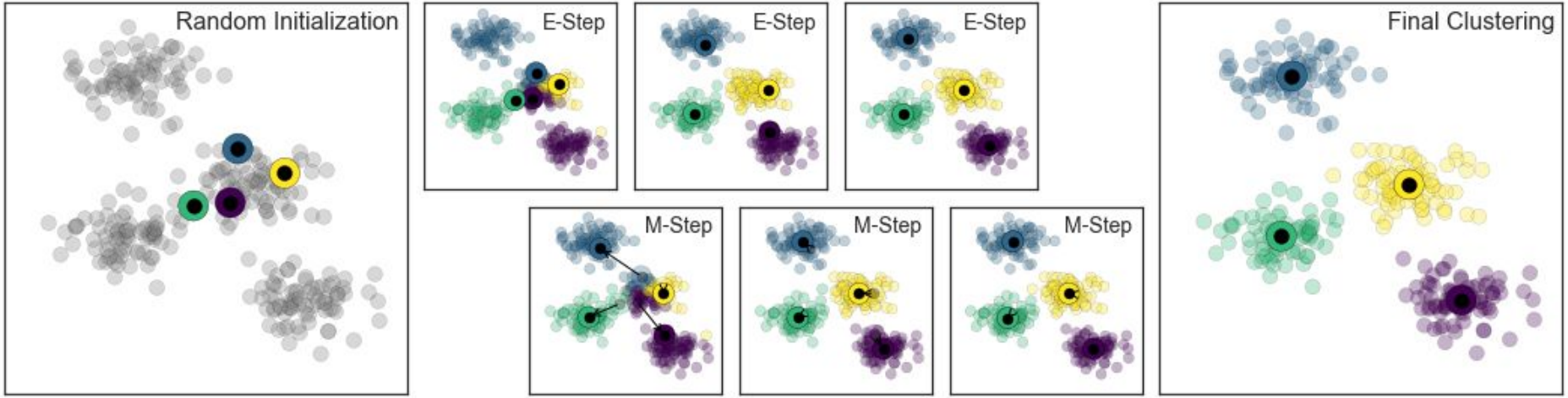


Image credit: https://jakevdp.github.io/PythonDataScienceHandbook/05.11-k-means.html

Example: k-Means

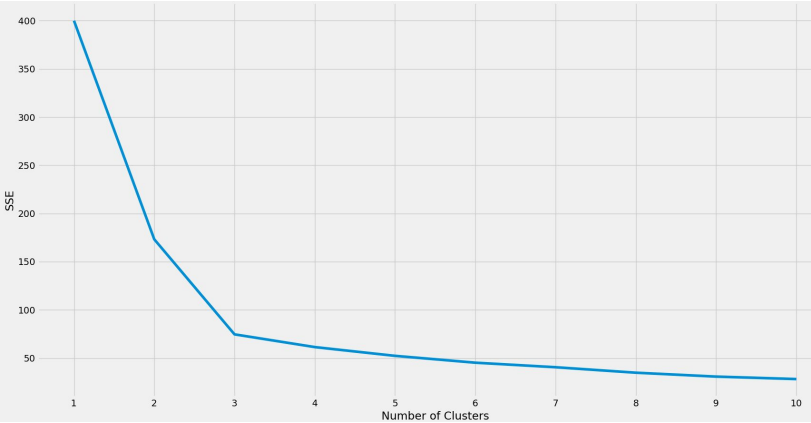
```python
# To install kneed in anaconda: conda install -c conda-forge kneed --yes
import matplotlib.pyplot as plt
from kneed import KneeLocator
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
feat,true_labels=make_blobs(n_samples=200,centers=3,cluster_std=2.75, random_state=42)
scaler = StandardScaler()
scaled_features = scaler.fit_transform(feat)
kmeans = KMeans(init="random", n_clusters=3, n_init=10, max_iter=300, random_state=42)
kmeans.fit(scaled_features)
print("SSE: ",kmeans.inertia_)                    # The lowest SSE value
print("centroids: ", kmeans.cluster_centers_)     # Final locations of the centroid
print("n_iter: ", kmeans.n_iter_)        # The number of iterations required to converge
print("First 5 labels: ", kmeans.labels_[:5])     # The first five predicted labels
```

Reference:
https://realpython.com/k-means-clustering-python/

```
KMeans(init='random', n_clusters=3, n_init=10, random_state=42)
SSE:  74.57960106819853
centroids:  [[ 1.19539276  0.13158148] [-0.25813925  1.05589975] [-0.91941183 -1.18551732]]
n_iter:  10
First 5 labels:  [0 1 2 2 2]
```

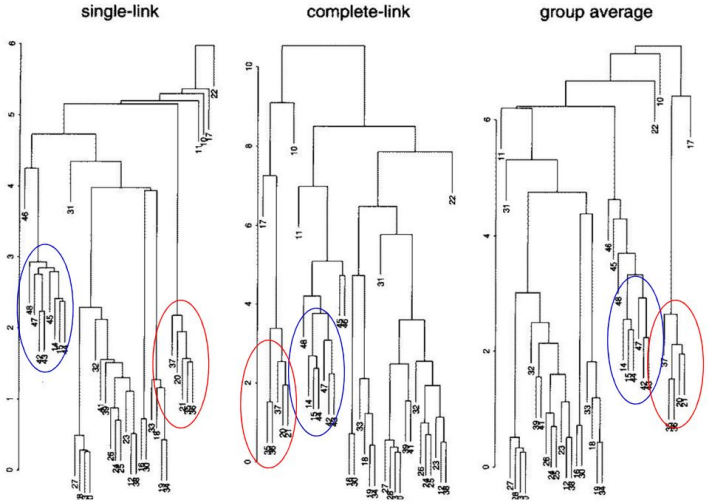
Example: k-Means - Determine the best k

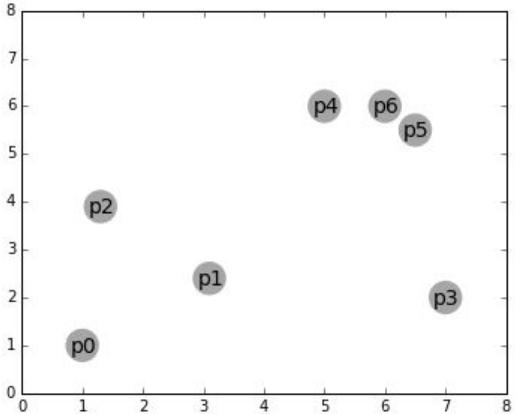
```
kmeans_kwargs={"init": "random", "n_init": 10, "max_iter": 300, "random_state": 42,}
sse = [] # A list holds the SSE values for each k
for k in range(1, 11):
  kmeans = KMeans(n_clusters=k, **kmeans_kwargs)
  kmeans.fit(scaled_features)
  sse.append(kmeans.inertia_)
plt.style.use("fivethirtyeight")
plt.plot(range(1, 11), sse)
plt.xticks(range(1, 11))
plt.xlabel("Number of Clusters")
plt.ylabel("SSE")
plt.show()
kl = KneeLocator(range(1, 11), sse, curve="convex", direction="decreasing")
print(kl.elbow)   # elbow method: 3
```



Agglomerative Clustering

- We start out with each data point forming its own cluster and gradually merge clusters until all points have been gathered together in one big cluster.
- At each stage the methods fuse the groups of data which are closest.





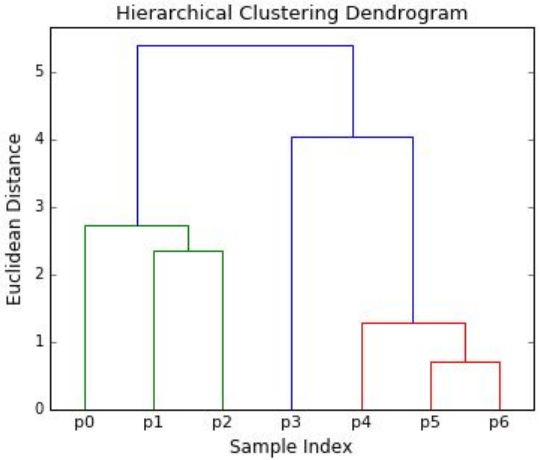


Image credit: Prof. CHAN Lai Wan,
https://dashee87.github.io/data%20science/general/Clustering-with-Scikit-with-GIFs/

Agglomerative Clustering Algorithms

1. Start with clusters of individual points and a proximity matrix
2. After some merging steps, we have some clusters
3. We want to merge the two closest clusters (e.g. C2 and C5) and update the proximity matrix
4. Then, update the proximity matrix accordingly
   - Single Linkage Clustering
   - Group Average Clustering
   - Complete Linkage Clustering
   - Ward Linkage (based on error sum of squares)
5. Until all clusters are merged into one

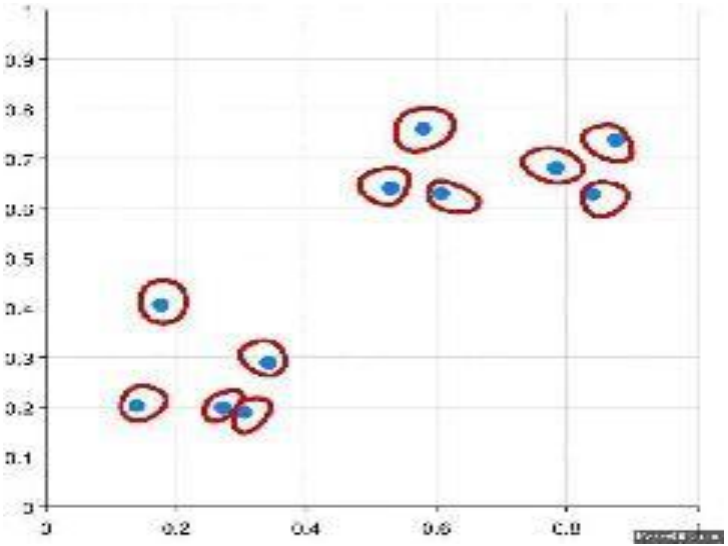
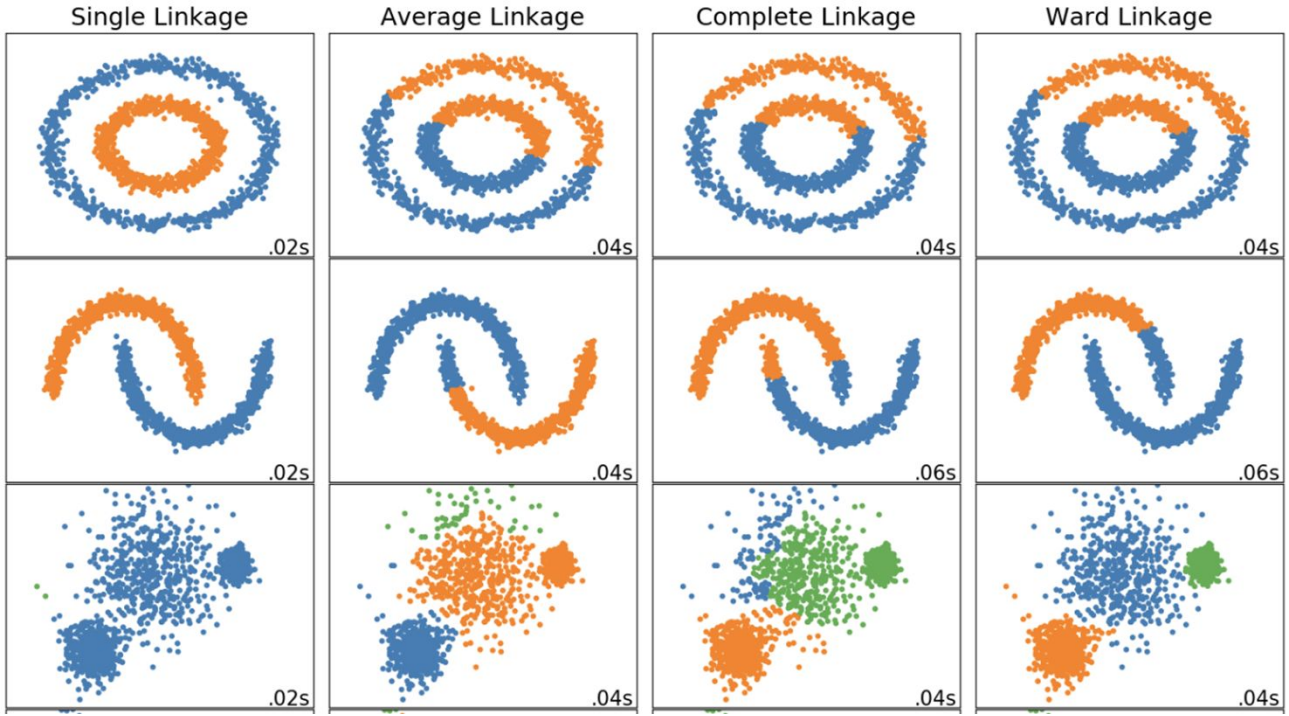


Image credit: Gfycat

Parameters in Agglomerative Clustering

1. Distance threshold
2. Number of clusters
3. The ways of updating the centroids



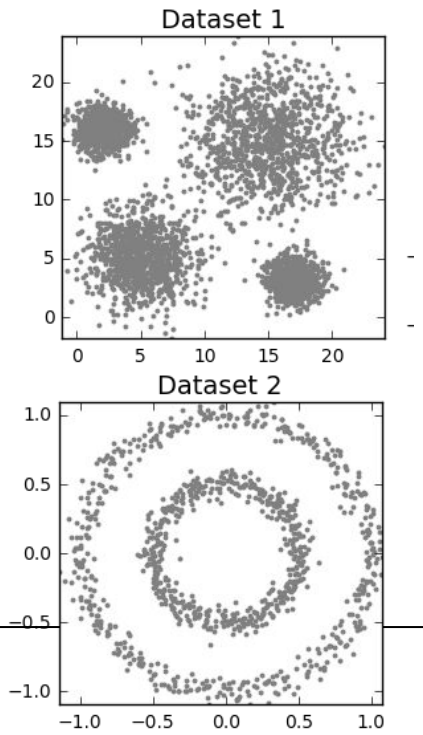
Example: AC - Synthetic Data

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score
from sklearn import cluster, datasets, mixture
from sklearn.neighbors import kneighbors_graph
np.random.seed(844)
clust1 = np.random.normal(5, 2, (1000,2))
clust2 = np.random.normal(15, 3, (1000,2))
clust3 = np.random.multivariate_normal([17,3], [[1,0],[0,1]], 1000)
clust4 = np.random.multivariate_normal([2,16], [[1,0],[0,1]], 1000)
dataset1 = np.concatenate((clust1, clust2, clust3, clust4))
# we take the first array as the second array has the cluster labels
dataset2 = datasets.make_circles(n_samples=1000, factor=.5, noise=.05)[0]
```

Example: AC - Visualize Data

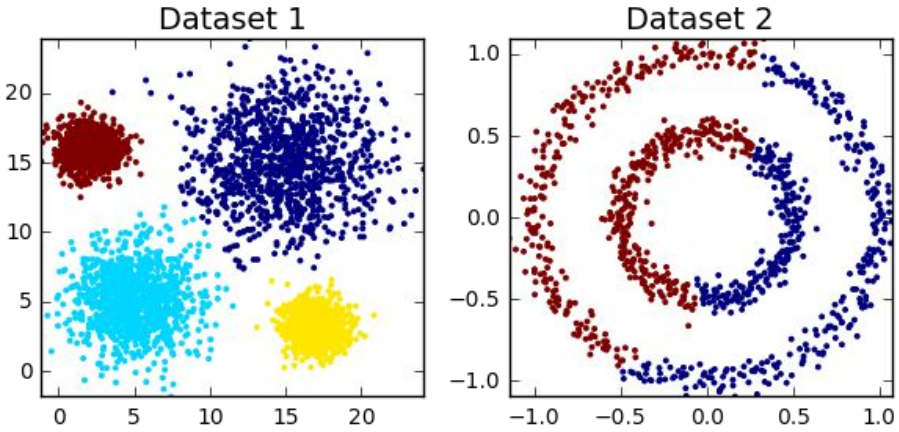
```
def cluster_plots(set1, set2, colours1 = 'gray', colours2 = 'gray', title1 = 'Dataset 1',  title2 = 'Dataset 2'):
    fig,(ax1,ax2) = plt.subplots(1, 2)
    fig.set_size_inches(6, 3)
    ax1.set_title(title1,fontsize=14)
    ax1.set_xlim(min(set1[:,0]), max(set1[:,0]))
    ax1.set_ylim(min(set1[:,1]), max(set1[:,1]))
    ax1.scatter(set1[:, 0], set1[:, 1],s=8,lw=0,c= colours1)
    ax2.set_title(title2,fontsize=14)
    ax2.set_xlim(min(set2[:,0]), max(set2[:,0]))
    ax2.set_ylim(min(set2[:,1]), max(set2[:,1]))
    ax2.scatter(set2[:, 0], set2[:, 1],s=8,lw=0,c=colours2)
    fig.tight_layout()
    plt.show()

cluster_plots(dataset1, dataset2)
```



Example: AC - Train and Use Model

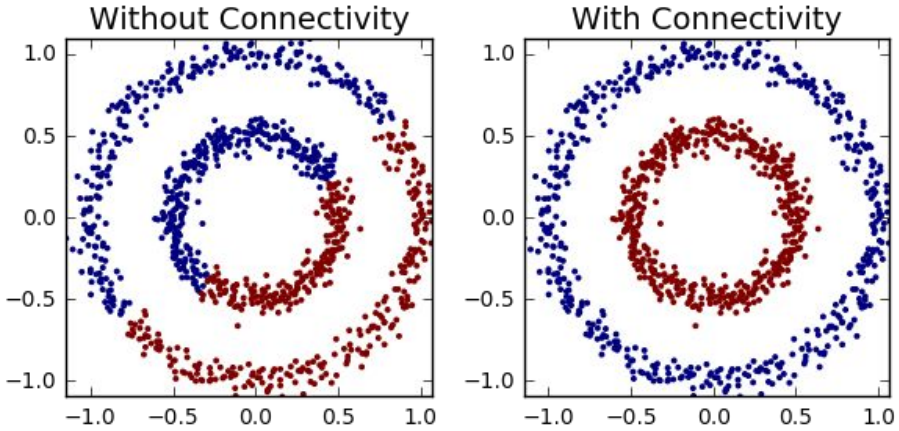
```
# implementing agglomerative (bottom up) hierarchical clustering
# we're going to specify that we want 4 and 2 clusters, respectively
hc_dataset1 = cluster.AgglomerativeClustering(n_clusters=4, metric='euclidean',
linkage='ward').fit_predict(dataset1)
hc_dataset2 = cluster.AgglomerativeClustering(n_clusters=2, metric='euclidean',
linkage='average').fit_predict(dataset2)
cluster_plots(dataset1, dataset2, hc_dataset1, hc_dataset2)
```



Reference: https://dashee87.github.io/data%20science/general/Clustering-with-Scikit-with-GIFs/

Example: AC - Using Connectivity Constraints

```
hc_dataset2 = cluster.AgglomerativeClustering(n_clusters=2, metric='euclidean',
linkage='complete').fit_predict(dataset2)
connect = kneighbors_graph(dataset2, n_neighbors=5, include_self=False)
hc_dataset2_connectivity = cluster.AgglomerativeClustering(n_clusters=2,
metric='euclidean', linkage='complete', connectivity=connect).fit_predict(dataset2)
cluster_plots(dataset2, dataset2, hc_dataset2, hc_dataset2_connectivity,
title1='Without Connectivity', title2='With Connectivity')
```



Reference: https://dashee87.github.io/data%20science/general/Clustering-with-Scikit-with-GIFs/

Spectral Clustering

Spectral clustering techniques make use of the **spectrum (eigenvalues)** of the similarity matrix (affinity matrix) of the data to perform **dimensionality reduction** before clustering in fewer dimensions. The similarity matrix is provided as an input and consists of a quantitative assessment of the relative similarity of each pair of points in the dataset.

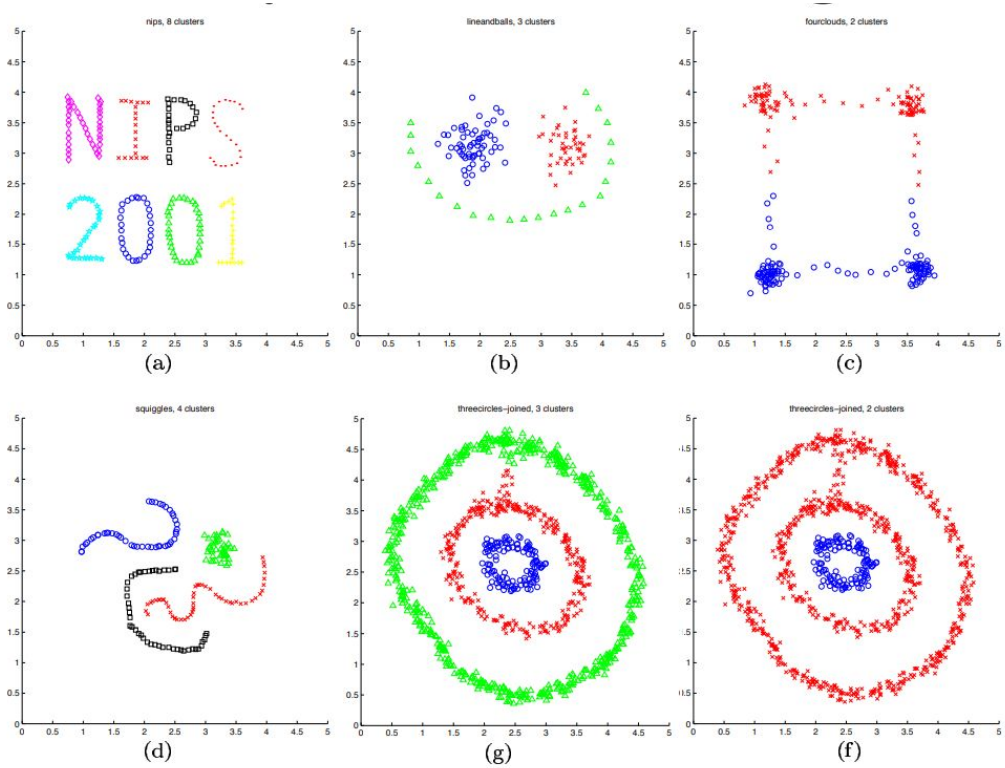


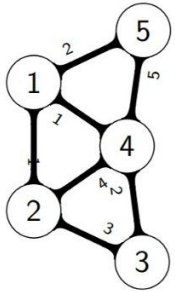
Image credit: Ng, Jordan, Weiss NIPS ‘01

Spectral Clustering Algorithms

1. Form a distance matrix
2. Transform the distance matrix into an affinity matrix A
3. Compute the degree matrix D and the Laplacian matrix L = D – A.
4. Find the eigenvalues and eigenvectors of L.
5. With the eigenvectors of k largest eigenvalues computed from the previous step form a matrix.
6. Normalize the vectors.
7. Cluster the data points in k-dimensional space.

$\mathbf{A}$	adjacency matrix
$\mathbf{W}$	weight matrix
$\mathbf{D}$	(diagonal) degree matrix
$\mathbf{L} = \mathbf{D} - \mathbf{W}$	graph **Laplacian** matrix

$$\mathbf{L} = \begin{pmatrix} 4 & -1 & 0 & -1 & -2 \\ -1 & 8 & -3 & -4 & 0 \\ 0 & -3 & 5 & -2 & 0 \\ -1 & -4 & -2 & 12 & -5 \\ -2 & 0 & 0 & -5 & 7 \end{pmatrix}$$



Reference: Analytics VIdhya

Graph Partitioning

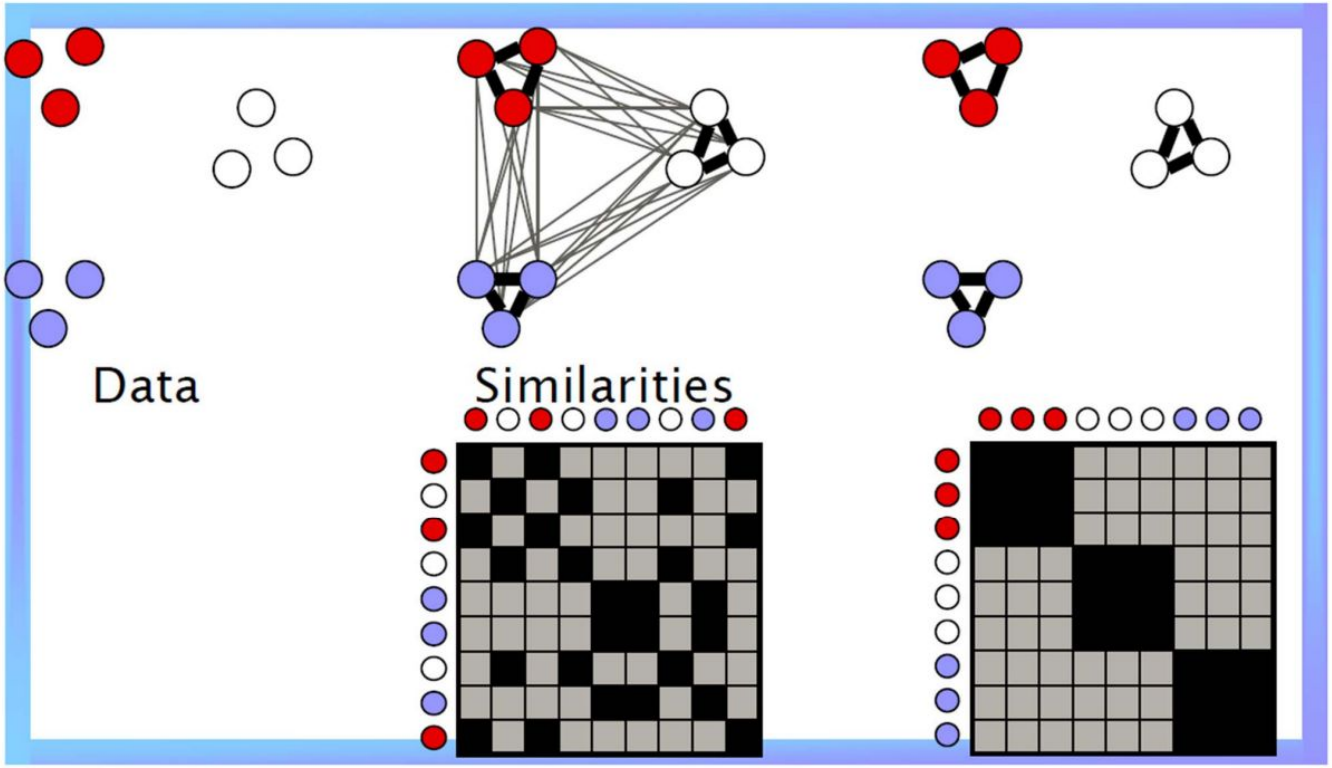
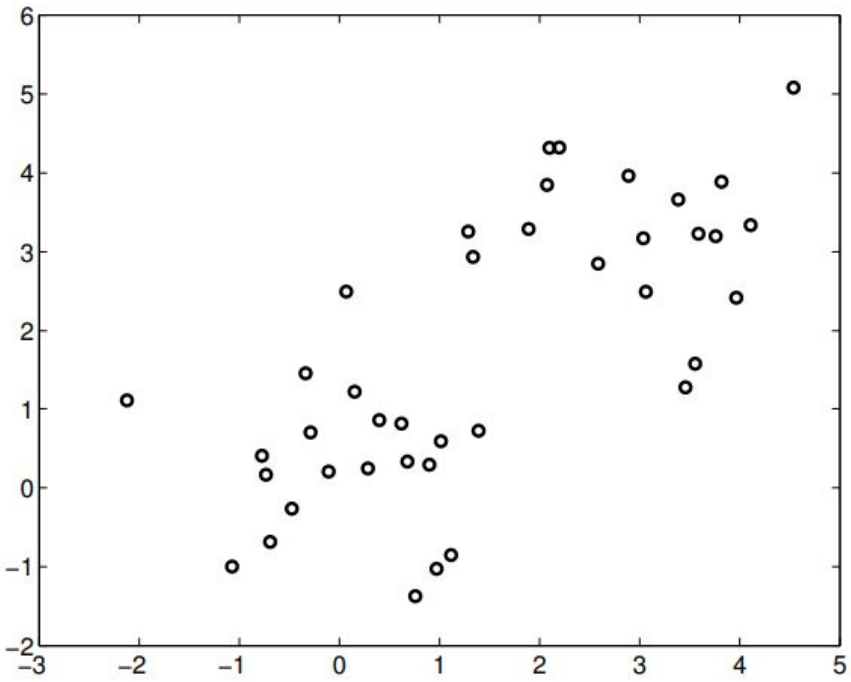


Image credit: http://people.csail.mit.edu/dsontag/courses/ml13/slides/lecture16.pdf

Visualization: Spectral Clustering



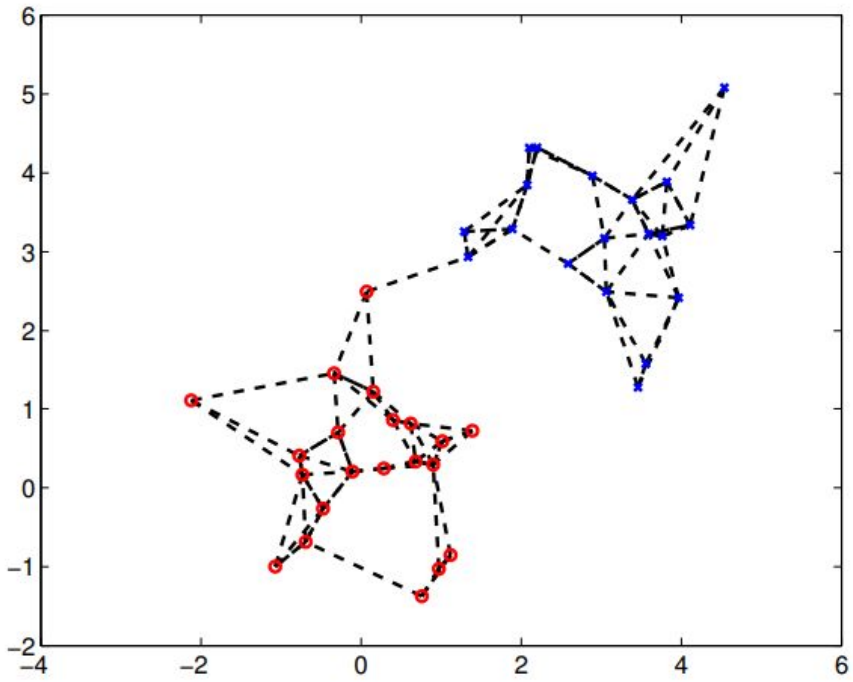


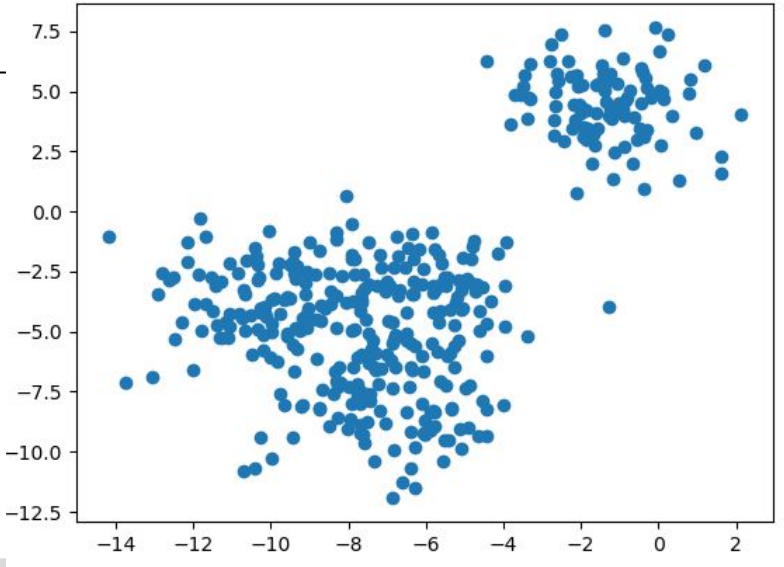
Image credit: http://people.csail.mit.edu/dsontag/courses/ml13/slides/lecture16.pdf

Parameters in Spectral Clustering

1. Number of clustering
2. Number of eigenvectors to use
3. Number of times of executing the k-means algorithms
4. Gamma, i.e. Kernel coefficient for rbf, poly, sigmoid, laplacian and chi2 kernels
5. Affinity, i.e the ways of constructing the affinity matrix

Example: SC - Load Data

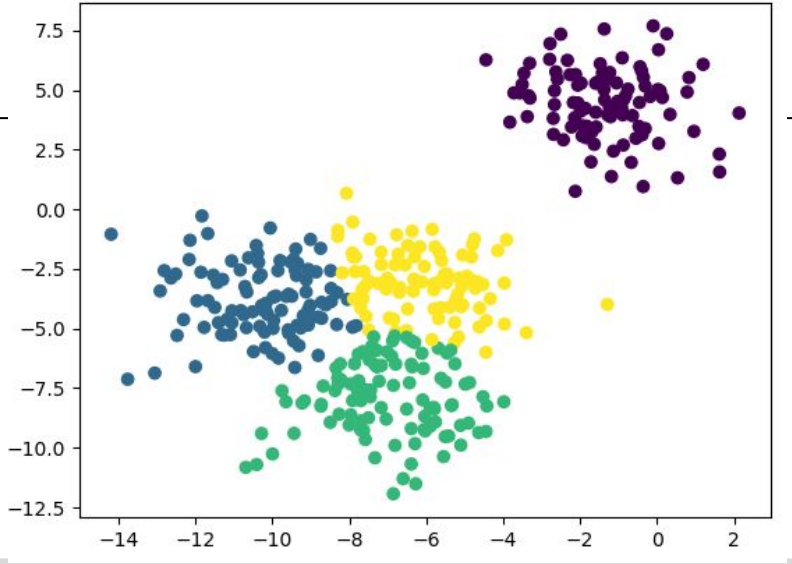
```python
from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from numpy import random
random.seed(1)
x, _ = make_blobs(n_samples=400,centers=4, cluster_std=1.5)
plt.scatter(x[:,0], x[:,1])
plt.show()
```



Reference:
https://www.datatechnotes.com/2020/12/spectral-clustering-example-in-python.html

Example: SC - Build and Use Model

```
sc = SpectralClustering(n_clusters=4).fit(x)
print(sc)
# SpectralClustering(n_clusters=4)
SpectralClustering(affinity='rbf', assign_labels='kmeans', coef0=1, degree=3,
eigen_solver=None, eigen_tol=0.0, gamma=1.0, kernel_params=None, n_clusters=4,
n_components=None, n_init=10, n_jobs=None, n_neighbors=10, random_state=None)
labels = sc.labels_
plt.scatter(x[:,0], x[:,1], c=labels)
plt.show()
```

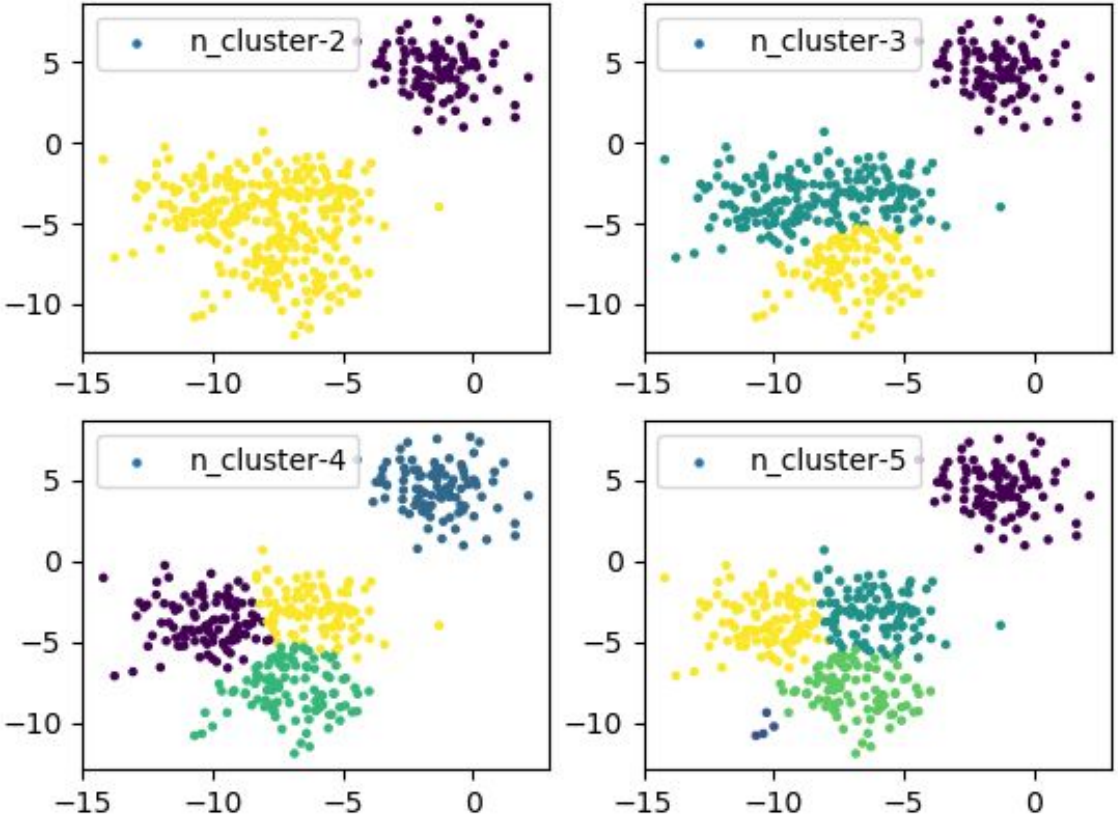


Example: SC - Visualize Different #Clusters

```
f = plt.figure()
f.add_subplot(2, 2, 1)
for i in range(2, 6):
    sc = SpectralClustering(n_clusters=i).fit(x)
    f.add_subplot(2, 2, i-1)
    plt.scatter(x[:,0], x[:,1], s=5, c=sc.labels_, label="n_cluster-"+str(i))
    plt.legend()

plt.show()
```

Example: SC - Visualize Different #Clusters



More about Different Clustering Algorithms

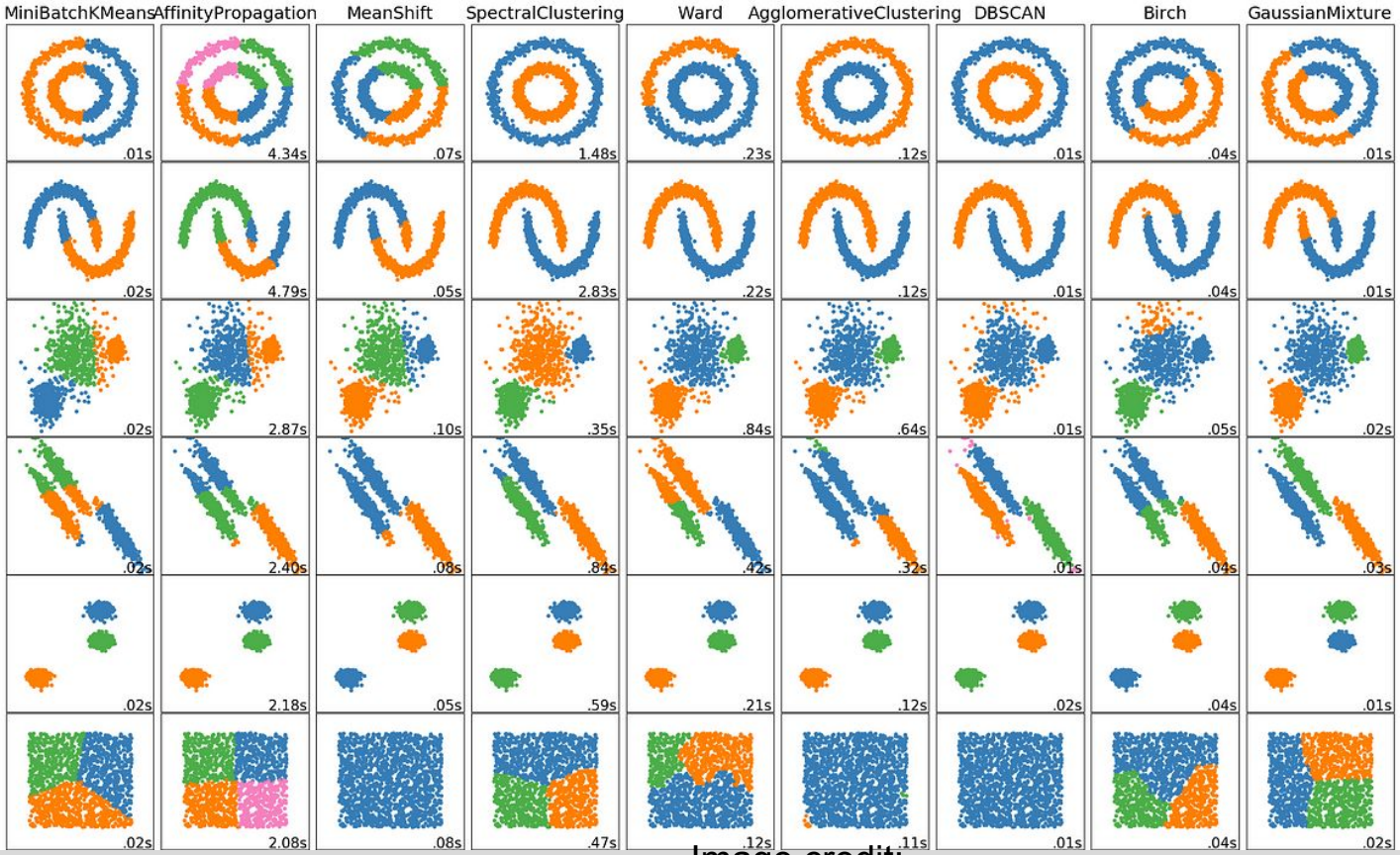


Image credit: https://scikit-learn.org/stable/modules/clustering.html

Metrics for Clustering Algorithms

- Clustering problems do not have any ground truth (class labels).
- However, we still wish to assess the quality of the clusters
- The three commonly used evaluation metrics:

1. **Silhouette score**: The separation distance between the clusters
2. **Calinski Harabasz index**: The ratio between two parameters within-cluster diffusion and between cluster dispersion.
3. **Davies-Bouldin index**: The principle of within-cluster and between-cluster distances.

Silhouette Score

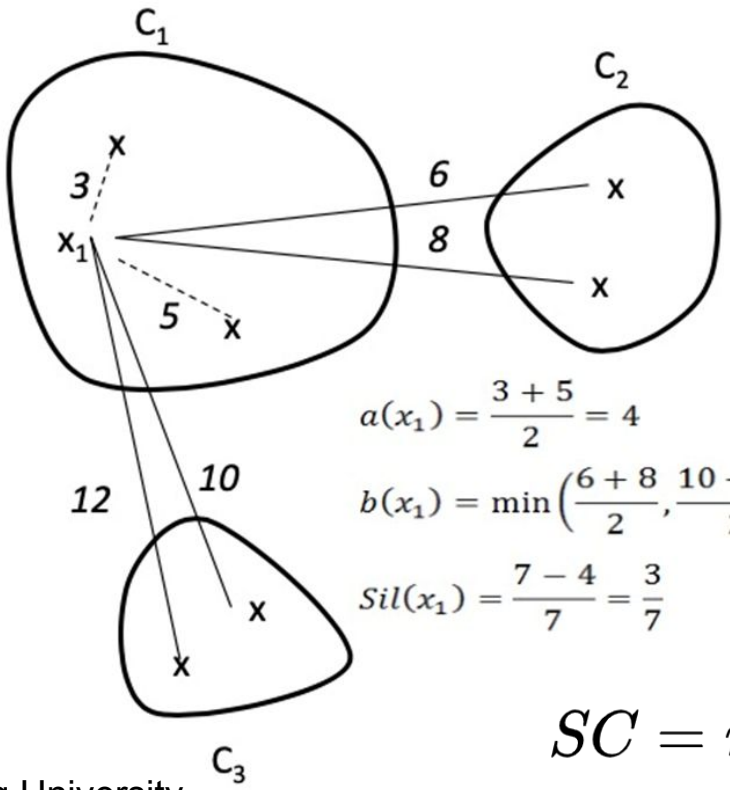
- Scores any clustering with an arbitrary number of unique clusters. Clustering can come from any clustering algorithm.
- **a(i)** = average dissimilarity of instance i to all other instances in the cluster to which i is assigned – Want it **small**
  - Dissimilarity could be Euclidean distance, etc.
- **b(i)** = the smallest average dissimilarity of instance i to all instances in the closest cluster to b(i) – Want it **large**
- b(i) is smallest for the best different cluster that i could be assigned to – the best cluster that you would move i to if needed

$$s(i) = \begin{cases} 1 - a(i)/b(i), & \text{if } a(i) < b(i) \\ 0, & \text{if } a(i) = b(i) \\ b(i)/a(i) - 1, & \text{if } a(i) > b(i) \end{cases}$$

$$-1 \leq s(i) \leq 1$$

$$SC = \max_k \tilde{s}(k) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

Example: Silhouette Score



$$a(x_1) = \frac{3+5}{2} = 4$$

$$b(x_1) = \min\left(\frac{6+8}{2}, \frac{10+12}{2}\right) = 7$$

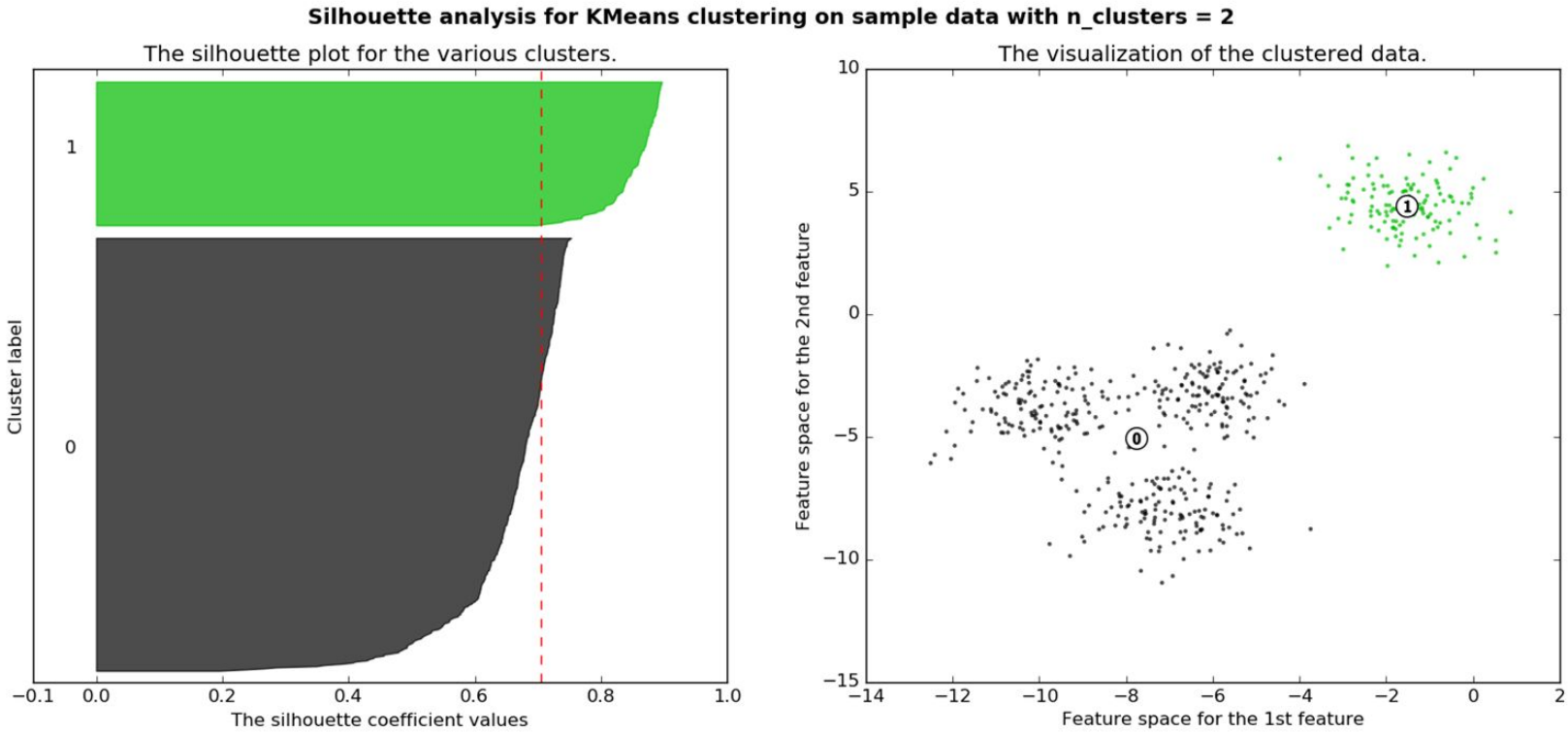
$$Sil(x_1) = \frac{7-4}{7} = \frac{3}{7}$$

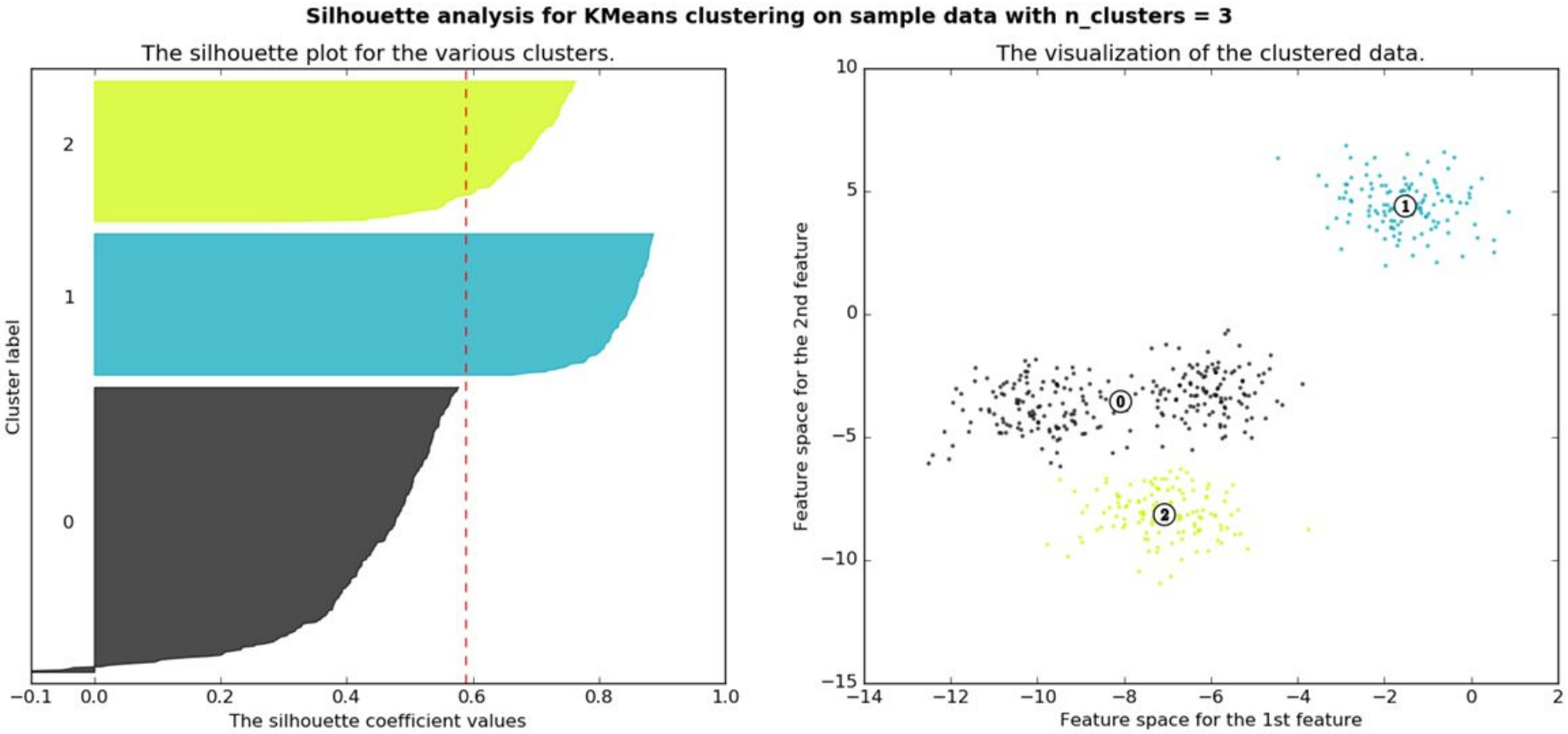
$$SC = \max_k \tilde{s}(k) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

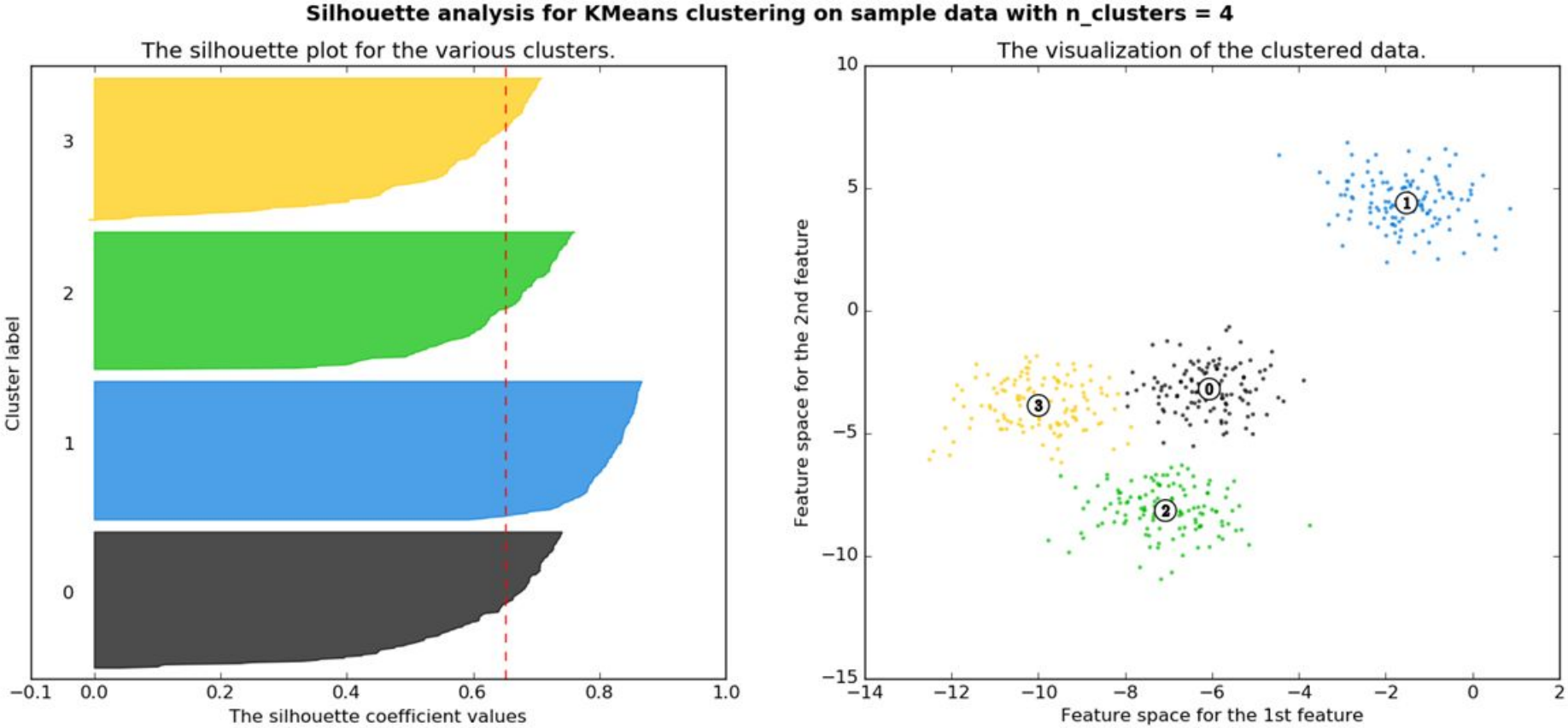
Reference: Brigham Young University

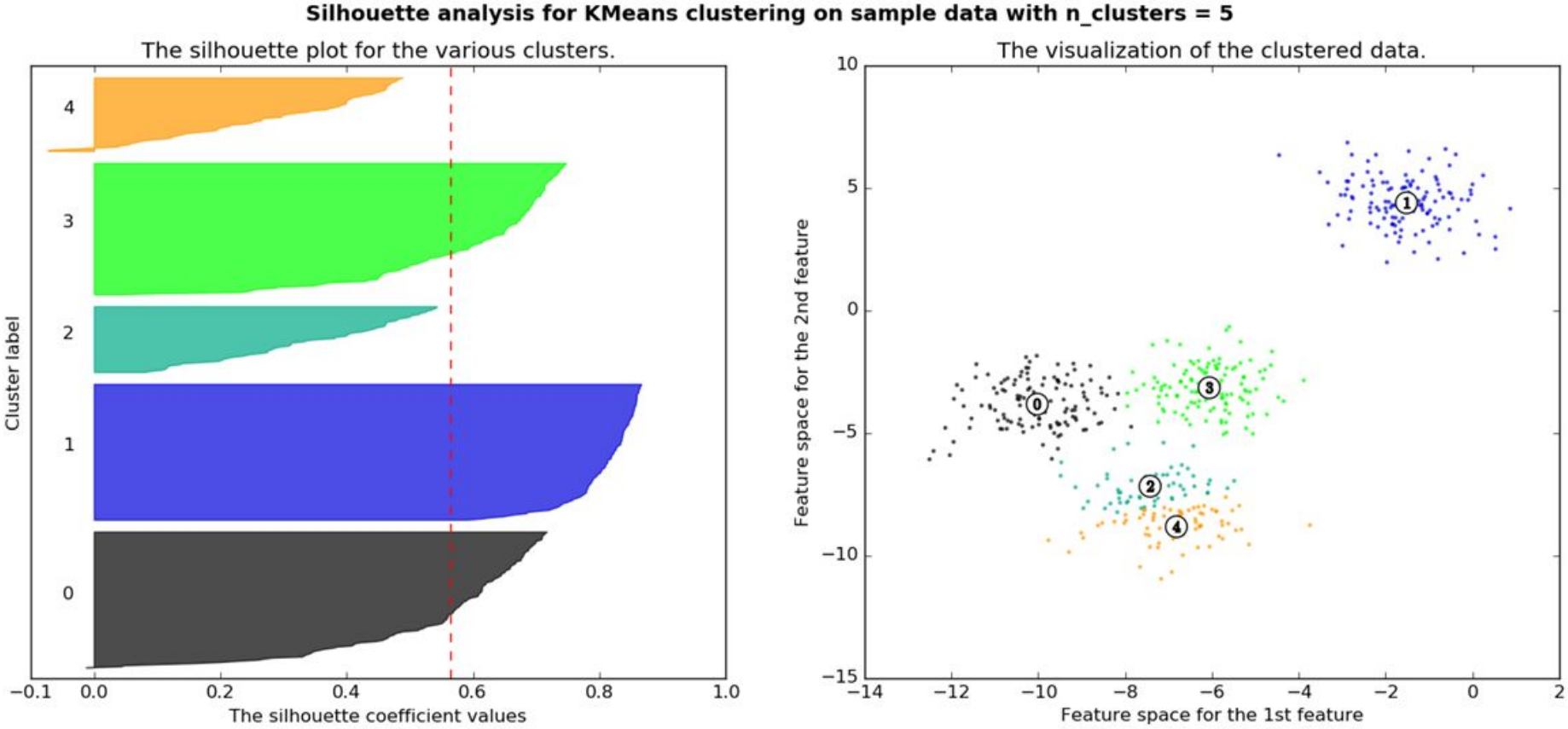
Visualizing Silhouette

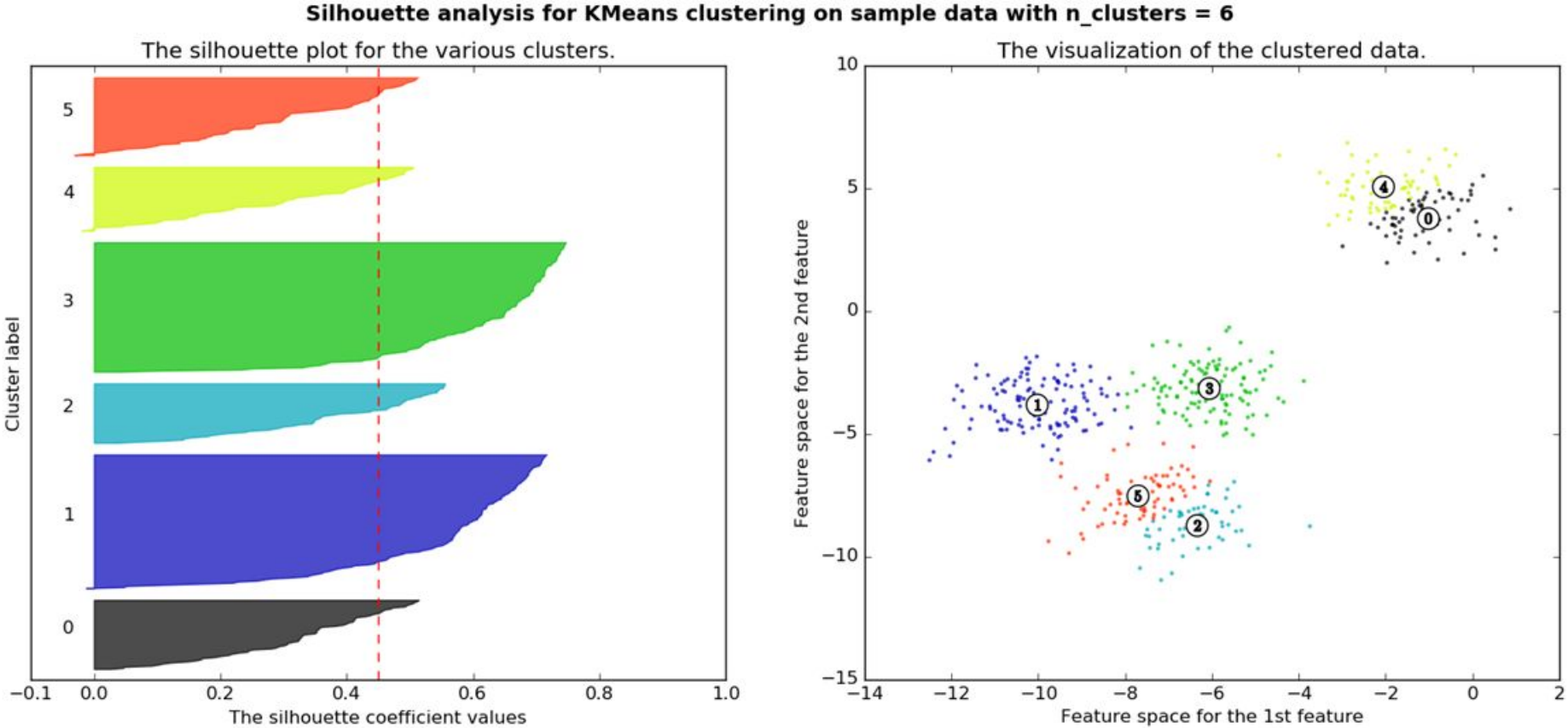
- Width is quality of cluster, height is size of cluster
- Dashed line is average silhouette score for clustering




Silhouette analysis for KMeans clustering on sample data with n_clusters = 3
The silhouette plot for the various clusters.
Cluster label
2
1
0
−0.1
0.0
0.2
0.4
0.6
0.8
1.0
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
10
5
0
−5
−10
−15
−14
−12
−10
−8
−6
−4
−2
0
2
Feature space for the 1st feature
1
0
2


Silhouette analysis for KMeans clustering on sample data with n_clusters = 4
The silhouette plot for the various clusters.
Cluster label
3
2
1
0
−0.1
0.0
0.2
0.4
0.6
0.8
1.0
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
10
5
0
−5
−10
−15
−14
−12
−10
−8
−6
−4
−2
0
2
Feature space for the 1st feature
1
0
3
2


Silhouette analysis for KMeans clustering on sample data with n_clusters = 5
The silhouette plot for the various clusters.
Cluster label
4
3
2
1
0
−0.1
0.0
0.2
0.4
0.6
0.8
1.0
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
10
5
0
−5
−10
−15
−14
−12
−10
−8
−6
−4
−2
0
2
Feature space for the 1st feature
1
0
3
2
4


Silhouette analysis for KMeans clustering on sample data with n_clusters = 6
The silhouette plot for the various clusters.
Cluster label
5
4
3
2
1
0
−0.1
0.0
0.2
0.4
0.6
0.8
1.0
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
10
5
0
−5
−10
−15
−14
−12
−10
−8
−6
−4
−2
0
2
Feature space for the 1st feature
4
0
1
3
5
2

Example: Silhouette Score

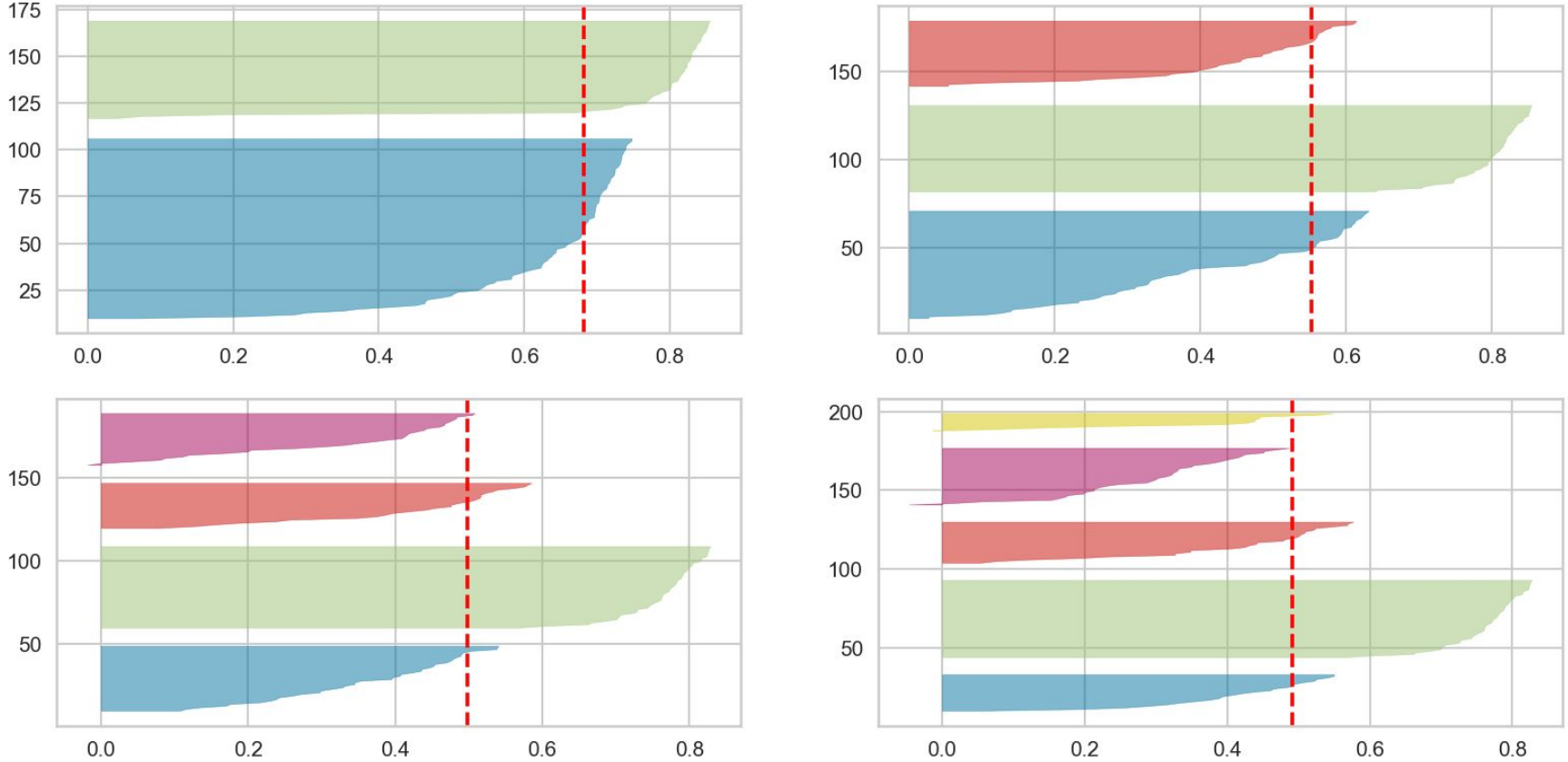
```python
from sklearn.metrics import silhouette_score
from sklearn import datasets
from sklearn.cluster import KMeans
iris = datasets.load_iris()
X = iris.data
y = iris.target
km = KMeans(n_clusters=3, init='k-means++', max_iter=300, n_init=10, random_state=0)
km.fit_predict(X)
score = silhouette_score(X, km.labels_, metric='euclidean')
print('Silhouetter Score: %.3f' % score)
import matplotlib.pyplot as plt
# installation: conda install -c districtdatalabs yellowbrick --yes
from yellowbrick.cluster import SilhouetteVisualizer
fig, ax = plt.subplots(2, 2, figsize=(15,8))
for i in [2, 3, 4, 5]:
  km = KMeans(n_clusters=i,init='k-means++',n_init=10,max_iter=100,random_state=42)
  q, mod = divmod(i, 2)
  visualizer = SilhouetteVisualizer(km, colors='yellowbrick', ax=ax[q-1][mod])
  visualizer.fit(X)
```

Larger value of Silhouette score means the clusters are better defined.

```
Silhouetter Score: 0.553
```

Example: Silhouette Score

Width is quality of cluster, height is size of cluster



Calinski Harabasz Index

- Calinski Harabasz Index is the validation of how well the clustering has been done is made using quantities and features inherent to the dataset.
- The ratio between two parameters within-cluster diffusion and between cluster dispersion
- The CH index for K number of clusters on a dataset D =[d_1, d_2, d_3, …, d_N] is defined as,

$$CH = \left[\frac{\sum_{k=1}^{K} n_k \|c_k - c\|^2}{K-1} \right] / \left[\frac{\sum_{k=1}^{K} \sum_{i=1}^{n_k} \|d_i - c_k\|^2}{N-K} \right]$$

n_k and c_k are the number of points and centroid of the k-th cluster respectively
c is the global centroid, N is the total no. of data points

Example: Calinski Harabasz Index

```python
from sklearn import datasets
from sklearn.cluster import KMeans
from sklearn import metrics
from sklearn.metrics import pairwise_distances
import numpy as np
# loading the dataset
X, y = datasets.load_iris(return_X_y=True)
# K-Means
kmeans = KMeans(n_clusters=3, random_state=1).fit(X)
# we store the cluster labels
labels = kmeans.labels_
print(metrics.calinski_harabasz_score(X, labels))
```

```
561.62775662962
```

Higher value of CH index means the clusters are dense and well separated

Davies-Bouldin Index

- Davies-Bouldin Index is the validation of how well the clustering has been done is made using quantities and features inherent to the dataset.
- The principle of within-cluster and between-cluster distances
- $R_{i,j}$ be a measure of how good the clustering scheme is
- $M_{i,j}$ the separation between the i-th and the j-th cluster
- S_i, the within cluster scatter for the i-th cluster

$$R_{i,j} = \frac{S_i + S_j}{M_{i,j}}$$

1. $R_{i,j} \geqslant 0.$
2. $R_{i,j} = R_{j,i}.$
3. When $S_j \geqslant S_k$ and $M_{i,j} = M_{i,k}$ then $R_{i,j} > R_{i,k}.$
4. When $S_j = S_k$ and $M_{i,j} \leqslant M_{i,k}$ then $R_{i,j} > R_{i,k}.$

$$D_i \equiv \max_{j \neq i} R_{i,j}$$

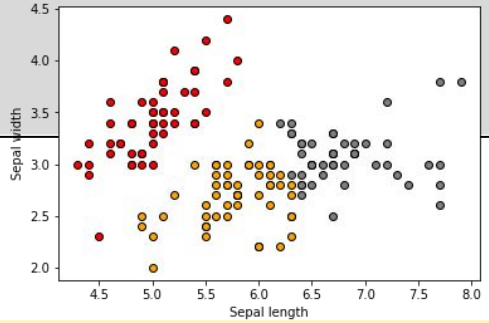
$$DB \equiv \frac{1}{N} \sum_{i=1}^{N} D_i$$

Reference: https://pyshark.com/davies-bouldin-index-for-k-means-clustering-evaluation-in-python/

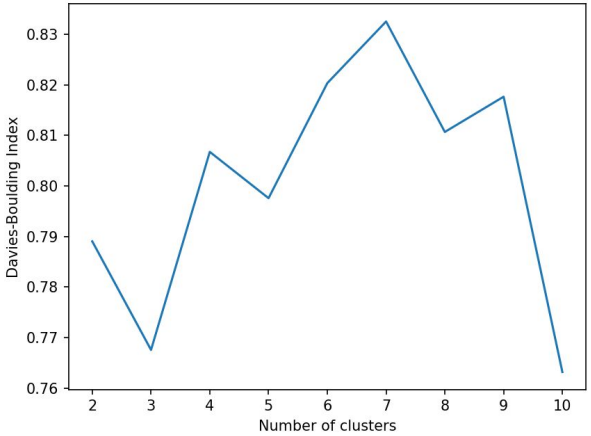
Example: Davies-Bouldin Index

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import davies_bouldin_score
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data[:, :2]
kmeans = KMeans(n_clusters=3, random_state=30)
labels = kmeans.fit_predict(X)
db_index = davies_bouldin_score(X, labels)
print(db_index)
results = {}
for i in range(2,11):
    kmeans = KMeans(n_clusters=i, random_state=30)
    labels = kmeans.fit_predict(X)
    db_index = davies_bouldin_score(X, labels)
    results.update({i: db_index})

plt.plot(list(results.keys()), list(results.values()))
plt.xlabel("Number of clusters")
plt.ylabel("Davies-Boulding Index")
plt.show()
```



Smaller value of DB index means the clusters are better defined.

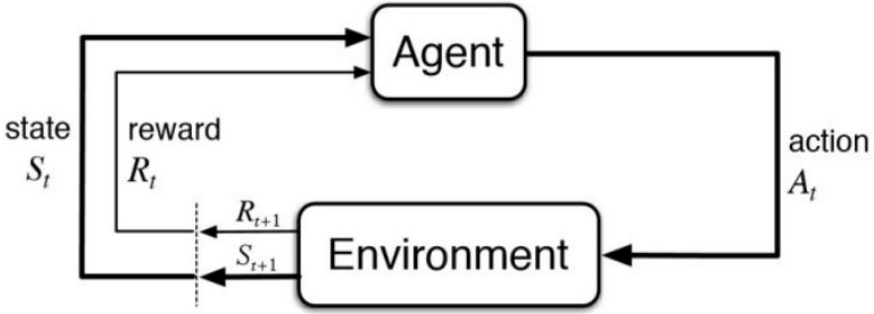



```
0.7675522686571647
```

Reinforcement Learning

Introduction to Reinforcement Learning

- Machine learning through interactions in simulation



- A computational approach to learning whereby **an agent** tries to **maximize** the total amount of **reward** it receives while interacting with a complex and uncertain **environment**.
  - Reference: Sutton and Barto Book

Brief Introduction to RL: https://www.youtube.com/watch?v=JgvyzIkgxF0

Agent and Environment

- The room is called the **environment** with which our agent interacts. It provides challenges (tempting furniture) and the desired objective (a satisfying scratching post).
- Two main types of environments in RL

1. **Discrete environments**: If the room turned into a classic arcade game with a grid, Bob could only move in four directions. Such environments have a limited number of options for both Bob's actions and the variations of the room.
2. **Continuous environments**: A real-world room allows nearly infinite possibilities to arrange the scratching posts and furniture. Besides, Bob is free to do anything cats usually do every day. That's why our room is a continuous environment with endless possibilities for Bob and the room.

States

- State space represents all possible states (situations) in which the agent and the environment find themselves at any given moment.
- The size of the state space depends on the environment type:

1. **Finite state space**: When the environment is discrete, the number of possible states is finite or countably infinite. Examples are board games, grid-like video games, or tic-tac-toe.
2. **Infinite state space**: When the environment is continuous, its state space is unbounded. Examples are video games with complex characters and environments, robots learning to walk, or in our case, cats learning to use a scratching post.

Reference: https://www.datacamp.com/tutorial/reinforcement-learning-python-introduction

Actions

- Action space is similar to the state space, the number of actions that the agent can take depends on the environment:

1. **Discrete Environments**: In a grid-world game, Bob might have a limited number of actions, like moving up, down, left, or right and scratching.
2. **Continuous Environments**: In a real-world room, Bob might have a broader range of actions, like moving in any direction, jumping, or even (hopefully not) chewing on wires.

Rewards, Time Steps, and Episodes

- For the agent to achieve its overall goal, it needs incentives or **rewards**. Most RL problems have pre-defined rewards. For example, in chess, capturing a piece is a positive reward, while receiving a check is a negative reward.
- To measure agent's learning journey's progress, we can think of its actions in terms of **time steps**. For instance, at time step t_1, Bob takes action a_1, which results in a new state s_1 (s_0 was the default state). It may also receive a reward r_1.
- A collection of time steps is called an **episode**. An episode always starts in a default state (the furniture and posts are set up) and **terminates** when the **objective is reached** (a post is found) or **the agent fails** (scratches furniture). Sometimes, an episode may also terminate based on **how much time has passed** (like in chess).

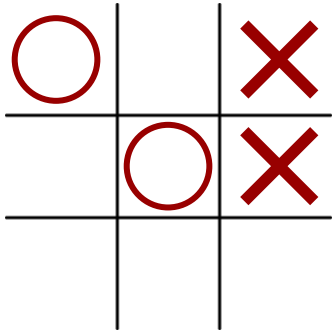
Policy

- The sum of the rewards an agent receives is called the **objective**.
- The agent's goal is to maximize the objective. It does this by selecting good actions. The function that an agent uses to decide on an action is called a **policy**.
- A policy π maps a state s to an action a: $\pi(s_0) = a_0$
- A policy can be stochastic (random) such that it may probabilistically output different actions for the same state.
- In this case it is expressed as an action sampled (indicated by ~) from a policy: $a \sim \pi(s)$

Reference: https://medium.com/@ZoinerTejada/ai-deep-reinforcement-learning-d317a4310d19

Example: Tic Tac Toe

- Tic-tac-toe is a two-player game who take turns marking the spaces in a three-by-three grid, one with Xs and the other with Os.
- **Goal**: Connect your token in a line (row / column / diagonal)
- **State**: Game board
- **Action**: Place your token
- **Reward**: 1 if you win, -1 if you lose
- **Policy**: π(s): pick the best square among the available squares based on the knowledge of the agent



Steps in Building the Q-Learning Agent for Tic-Tac-Toe

1. Create the game environment
2. Create the class for Q-Learning Agent
   - Q-Learning uses Q-Table to store the knowledge of how to play the game.
   - In the Q-Table of Tic-Tac-Toe,
     i. Each page (row) shows a different board layout
     ii. The columns represent the 9 squares where you can play
     iii. The numbers (Q-Values) tell the expected future rewards - higher numbers mean better chances of winning later in the game
3. Train the Q-Learning Agent through playing Tic-Tac-Toe lots of times
4. Record the training history to assess the training performance

Code: https://colab.research.google.com/drive/1SJAya1KtAipzgd8PiTFQRi2lN3WGHJP8?usp=sharing

Q-Learning Updating Formula

$$Q^{new}(S_t, A_t) \leftarrow (1 - \underbrace{\alpha}_{\text{learning rate}}) \cdot \underbrace{Q(S_t, A_t)}_{\text{current value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{\left(\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}_{\text{new value (temporal difference target)}}$$

1. **Current Estimate**: The term Q(S,A) represents the agent's current estimate of the value of taking action aaa in state sss.
2. **Reward**: R represents the immediate reward received after taking action A.
3. **Future Estimate**: The term γmaxQ(S’,A) captures the estimated maximum future reward from the next state S’.
4. **Learning**: The whole expression represents the difference between the estimated future rewards and the current estimate, which is scaled by the learning rate α.

Image credit: Wikipedia

Example: Tic-Tac-Toe Agent

```python
class QLearningAgent:
    def __init__(self, alpha=0.1, gamma=0.9, epsilon=0.1):
        self.q_table = defaultdict(lambda: np.zeros(9))
        self.alpha = alpha      # learning rate
        self.gamma = gamma      # discount factor
        self.epsilon = epsilon  # exploration rate

    def get_action(self, state, available_moves):
        state_key = self._state_to_key(state)

        if random.random() < self.epsilon:
            # Explore: random move
            return random.choice(available_moves)
        else:
            # Exploit: best move from Q-table
            q_values = self.q_table[state_key]
            # Only consider available moves
            available_q_values = {move: q_values[move] for move in available_moves}
            return max(available_q_values, key=available_q_values.get)
```

Example: Tic-Tac-Toe Agent

```
    def update_q_value(self, state, action, reward, next_state, next_available_moves):
        state_key = self._state_to_key(state)
        next_state_key = self._state_to_key(next_state)

        current_q = self.q_table[state_key][action]

        if next_available_moves:
            # If game continues, use max Q-value of next state
            next_q_values = self.q_table[next_state_key]
            next_available_q_values = [next_q_values[move] for move in next_available_moves]
            max_next_q = max(next_available_q_values) if next_available_q_values else 0
        else:
            # If game over, no future rewards
            max_next_q = 0

        # Q-learning formula
        new_q = (1-self.alpha)*current_q + self.alpha*(reward + self.gamma*max_next_q)
        self.q_table[state_key][action] = new_q
```

Example: Tic-Tac-Toe Agent

```
    def _state_to_key(self, state):
        # Convert state to a consistent key format
        if isinstance(state, tuple):
            return state
        return tuple(state)

```

Exploration and Exploitation

- Agent only experiences what happens for the actions it tries!
- How should an RL agent balance its actions?
  - **Exploration**: trying new things that might enable the agent to make better decisions in the future
  - **Exploitation**: choosing actions that are expected to yield good reward given past experience
- Often there may be an exploration-exploitation tradeoff
  - May have to sacrifice reward in order to explore and learn about potentially better policy
- In the Tic-Tac-Toe example,
  - Exploration: Random move
  - Exploitation: Decision made based on the Q-Table

Example: Exploration and Exploitation

1. **Restaurant Selection**
   - Exploitation: Go to your favourite restaurant
   - Exploration: Try a new restaurant
2. **Online Banner Advertisements**
   - Exploitation: Show the most successful advert
   - Exploration: Show a different advert
3. **Oil Drilling**
   - Exploitation: Drill at the best known location
   - Exploration: Drill at a new location
4. **Game Playing**
   - Exploitation: Play the move you believe is
   - Exploration: play an experimental move

Example: Exploration and Exploitation

- **Epsilon-greedy learning**: Let’s imagine for a second that the agent has a special “scratchy-meter” that generates random numbers. If the generated number is smaller than some predefined threshold called epsilon, the agent tries a random scratching surface (exploration). But if the number is greater than epsilon, the agent goes for the post that felt best before (exploitation).
- **Boltzmann exploration**: If the agent keeps scratching things that don't feel right (getting negative rewards), it's more likely to explore new options (increased exploration). But as the agent finds the perfect scratching post (positive rewards), he'll stick to that happy spot (exploitation).

Example: Tic-Tac-Toe Agent Training Loop Version 1

```
def train_agent(episodes=10000):
    game = TicTacToe()
    agent = QLearningAgent()
    results = {'X': 0, 'O': 0, 'draw': 0}
    for episode in range(episodes):
        state = game.reset()
        # Randomly decide if agent plays as X or O
        agent_is_x = random.choice([True, False])

        while not game.game_over:
            available_moves = game.available_moves()
            if (agent_is_x and game.current_player == 'X') or (not agent_is_x and game.current_player == 'O'):
                # Agent's turn
                action = agent.get_action(state, available_moves)
                old_state = state
                game.make_move(action)
                new_state = game.get_state()

```

Example: Tic-Tac-Toe Agent Training Loop Version 1

```
                # Calculate reward
                if game.game_over:
                    if game.winner == game.current_player:  # Agent won
                        reward = 1
                    elif game.winner:  # Agent lost
                        reward = -1
                    else:              # Draw
                        reward = 0
                else:
                    reward = 0

                # Update Q-value
                agent.update_q_value(old_state, action, reward, new_state, game.available_moves())

            else:
                # Opponent's turn (random moves)
                action = random.choice(available_moves)
                game.make_move(action)
```

Example: Tic-Tac-Toe Agent Training Loop Version 1

```
            state = game.get_state()

        # Record results
        if game.winner == 'X':
            results['X'] += 1
        elif game.winner == 'O':
            results['O'] += 1
        else:
            results['draw'] += 1

        if (episode + 1) % 1000 == 0:
            print(f"Episode {episode + 1}: {results}")

    return agent
```

There are some issues in the Training Loop

1. Rewards are evaluated only when the game ends. There are no intermediate rewards designed for training. For example,
   - Negative: Opponent has winning move after my turn
   - Positive: I create a winning move
   - Negative: I fail to generate winning opportunities (except the first move)
   - … etc
2. We can train two Q-Learning agents together, instead of a Q-Learning agent VS a random agent.
   - First player has very huge advantages in playing Tic-Tac-Toe

Code fo version 2:
https://colab.research.google.com/drive/1h-KVK5234vwZ19JcXpa4smsPF1JnlEMp?usp=sharing

Evaluation for the Agent

1. The change of the win rate against a random agent during training
2. The win rate against a random agent during testing
3. The number of different moves that the agent can use, by exploring the Q-Table
4. The comment from the skilled opponents after some matches

Categorization of RL Algorithms

1. Model-Free VS Model-Based
   - **Model-Free** Methods: The algorithm does not learn a model of the environment. It learns a policy and/or value function directly from interaction with the environment.
     - E.g. Q-Learning, DQN, Policy Gradient methods, … etc
   - **Model-Based** Methods: The algorithm learns a model of the environment's dynamics (i.e., the transition probabilities and rewards).
     - E.g. AlphaZero, Dyna-Q, Model-Based Policy Optimization (MBPO), Dreamer, and Probabilistic Inference for Learning Control (PILCO), … etc

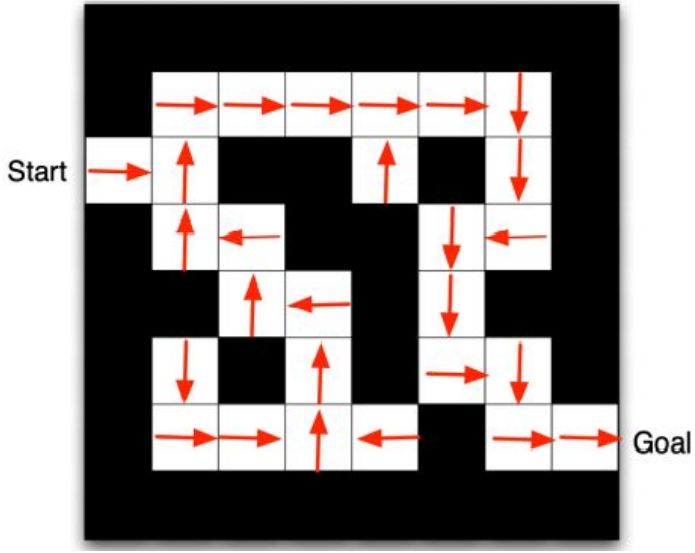
Categorization of RL Algorithms

2. Value-Based VS Policy-Based VS Actor-Critic
   - **Value-Based** Methods: The goal is to learn the optimal value function (V(s) or Q(s,a)). The policy is then derived directly from the value function.
     - E.g. Q-Learning, DQN, Dyna-Q, … etc
   - **Policy-Based** Methods: The goal is to directly learn the optimal policy $\pi(a|s)$ without learning a value function. They directly optimize the policy parameters.
     - E.g. Policy Gradient methods like REINFORCE, … etc
   - **Actor-Critic** Methods: A hybrid approach that combines the strengths of both.
     - E.g. PPO (Proximal Policy Optimization), … etc

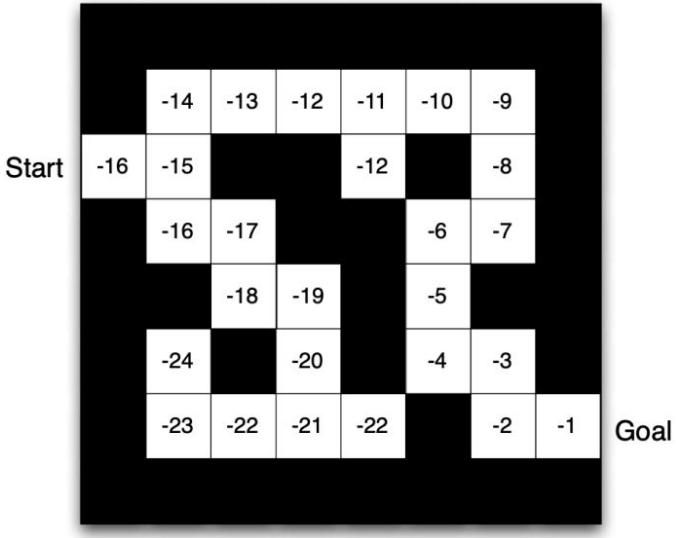
Example: Policy-based VS Value-based in Maze Exploration

- Arrows/numbers represent the policy/value for each state

Policy-based:



Value-based:



More reinforcement learning examples will be discussed in the case studies

What’s Next?

- We learnt the basic skills for machine learning algorithms
  - **Classification**, **Regression**, **Clustering**, **Reinforcement Learning**
- We learnt how to **evaluate** and compare different machine learning algorithms

- We are going to learn how to **see**, how to **visualize** the data